MODULE 3

Storage class

What is storage class

- There are two different ways to characterize a variable.
  - Data type
  - Storage class
- Data type refers to `int, float or char`
- Storage class are used to describe the features of a variable
- It defines the scope (visibility) and lifetime of variables

Storage class

It tells the compiler

- The storage area (memory location) of the variable
- The initial value of the variable ….if not initialized
- The scope of the variable (within a function or within a `for/if` block)
- The life time of the variable

Types of storage class

Scope of variable:

It indicates the region/block over which the variable's declaration has an effect or in other words that particular variable can be used.

Lifetime of variable:

It refers to the period during which a variable retains a given value during the execution of a program. It tells how long a variable would be *active* in the program.

Types of storage class

Scope of variable: It indicates the region over which the variable's declaration has an effect or in other words that particular variable can be used.

There are four types of storage class

- Automatic storage class (`auto`)
- Static storage class (`static`)
- External storage class (`extern`)
- Register storage class (`register`)

Different storage classes

Class	Storage	Scope	Default (Initial) Value	Lifetime
`auto`	RAM	Local	Garbage Value	Within a function
`extern`	RAM	Global	Zero	Till the main program ends. One can declare it anywhere in a program.
`static`	RAM	Local	Zero	Till the main program ends. It retains the available value between various function calls.
`register`	Register	Local	Garbage Value	Within the function

Automatic storage class

- Keyword is **`auto`**
- Variable declared inside a function has by default automatic storage class, if it is not explicitly specified.
- They are local to the function or block (loop) in which they are declared
- They can be only accessed within the block/ function they have been declared and not outside them
- It can be assigned initial value. If not, it assumes garbage value.

WAP to show the working of auto storage class

```
#include<stdio.h>
void main()
{
 int x = 10;
 printf("initial x = %d\n", x);
     {
     int x = 20;
     printf("Value of x in block = %d\n",x);
     }
 printf("Value of x after block = %d", x);
}
```

The declaration syntax:

`int x` or `auto int x;`

Output

initial x = 10

Value of x in block = 20

Value of x after block = 10

External storage class

- Keyword is **`extern`**
- These variables are available to all the functions
- Their scope extends from the point of definition through out the entire program
- They are declared outside the function body
- The main purpose of using extern variables is that they can be accessed between two different files which are part of a large program.

WAP to show the working of extern storage class

```
include<stdio.h>
void display();
extern int y;
int y = 40;
void main()
{
   int y = 50;
   printf("Value of y = %d\n", y);
   display();
}
void display()
{
   printf("Value of y =
}
```

The declaration syntax:
`extern int y;`

Output
Value of y = 50
Value of y in function = 40

Static storage class

The declaration syntax:

```
static int x;
```

- Keyword is **`static`**
- It may be either internal or external depending on the place of declaration
- They are defined within individual functions and therefore have the same scope as automatic variables.
- If a function is exited and then re-entered at a later time, the static variables defined within that function will return their former values.

WAP to show the working of static storage class

Static

```
#include<stdio.h>
void incr();
int main()
{
   incr();
   incr();
   incr();
}
void incr()
{
   static int i = 1;
   printf("i = %d\n",i);
   i=i+1;
}
```

Output
i = 1
i = 2
i = 3

Auto

```
#include<stdio.h>
void incr();
int main()
{
   incr();
   incr();
   incr();
}
void incr()
{
   auto int i = 1;
   printf("i
   i=i+1;
}
```

;

Output
i = 1
i = 1
i = 1

Register storage class

- Keyword **`register`**
- advises the compiler to store the variable in the CPU's register instead of RAM for faster access.
- However if a register is not free, `auto` storage class is used
- variable can have a maximum size equal to the register size, else `auto` is assumed
- CPU registers are accessed by the computer very fast and so the program is executed very fast. So it is best used for loop counters

WAP to show the working of register storage class

```
#include<stdio.h>
void func();
int main()
{
    func();
    func();
    return 0;
}
void func()
{
    register int x = 1;
    printf("x = %d\n", x);
    x = x+1;
}
```

The declaration syntax:

```
register int x;
```

Output

x = 1

x = 1